Università degli Studi di Roma Tor Vergata

Corso di Cybersecurity

Relazione di Progetto

HazChain
Web application per il tracciamento di rifiuti tossici su Blockchain

Componenti del gruppo

Mihai Alexandru Sandu	Matricola: 0378481
Matteo Sabelli	Matricola: 0382089

Contents

1 Introduzione **2**

2 Obiettivi del progetto **2**

3 Architettura del sistema **2**
3.1 Panoramica dei componenti . . . 3
3.2 Flusso delle richieste . . . 3
3.3 Interazione con la Blockchain API . . . 4
3.4 Modello dati su Blockchain . . . 4
3.5 Workflow operativo . . . 5

4 Struttura del codice **5**
4.1 Frontend . . . 5
4.2 Backend . . . 6

5 Interfacce utente **7**
5.1 Dashboard disponibili e responsabilità . . . 7
5.2 Pagina di autenticazione (Login/Registrazione) . . . 7
5.3 Sequenze principali da documentare . . . 8

6 Funzionalità del sistema HazChain **11**
6.1 Autenticazione utente (Registrazione e Login) . . . 11
6.1.1 Funzionamento algoritmico . . . 11
6.2 Creazione attori (`ACTOR`) . . . 12
6.3 Creazione pacco (`PACKAGE`) . . . 12
6.4 Richiesta ritiro Lab → Van (`TRANSFER: WAITING`) . . . 12
6.5 Pickup e trasporto Van → Hub (`TRANSFER: SHIPPING/COMPLETED`) . . . 12
6.6 Richiesta ritiro Hub → Truck (`TRANSFER: WAITING`) . . . 12
6.7 Pickup e trasporto Truck → Landfill (`TRANSFER: SHIPPING/COMPLETED`) . . . 12
6.8 Smaltimento finale (`PACKAGE: DISPOSED`) . . . 12
6.9 Inspector . . . 12
6.10 Storico del Package . . . 12
6.11 `ALERT` e regole di blocco . . . 13

7 Requisiti **13**
7.1 Requisiti funzionali . . . 13
7.2 Requisiti non funzionali e sicurezza . . . 14

8 Stakeholder **14**

9 Considerazioni finali e sviluppi futuri **15**

1 Introduzione

La gestione dei rifiuti tossici prodotti nei laboratori richiede un tracciamento preciso di tutto il percorso del rifiuto: dalla creazione del pacco, ai passaggi di custodia tra trasportatori e centri di raccolta, fino allo smaltimento finale. Spesso questo processo viene gestito con registri centralizzati o documenti cartacei, che possono essere soggetti a errori, perdita di informazioni o modifiche non autorizzate, rendendo più difficile effettuare controlli e verifiche.
Il progetto **HazChain** è una applicazione web che supporta il tracciamento della *chain-of-custody* dei rifiuti tossici usando una Blockchain come registro immutabile degli eventi. Ogni operazione importante (creazione pacco, richiesta di ritiro, presa in carico, trasferimento, consegna e smaltimento) viene salvata, così da garantire tracciabilità e integrità dei dati. Il sistema è utilizzabile tramite dashboard dedicate ai diversi ruoli coinvolti (laboratorio, van, hub, truck e discarica), oltre a una sezione di ispezione per consultare tutte le informazioni riguardanti i pacchi.
Un elemento chiave di **HazChain** è la presenza di controlli automatici prima di registrare un evento: ad esempio si verifica che la sequenza degli stati sia corretta, che il *riskCode* sia compatibile con l'attore che riceve il pacco e che i trasportatori rispettino i vincoli di capacità. Se una regola viene violata, l'operazione viene bloccata e viene registrato un `ALERT` su Blockchain, in modo che l'anomalia sia tracciabile e verificabile.
Questo documento descrive le specifiche presentando contesto e obiettivi, architettura e modello dati, interfacce utente e principali funzionalità.

2 Obiettivi del progetto

L'obiettivo principale è realizzare una piattaforma web per tracciare in modo chiaro e verificabile l'intero ciclo di vita dei rifiuti tossici prodotti dai laboratori di chimica, garantendo la ricostruzione completa della *chain-of-custody*.
Gli obiettivi specifici del progetto sono:

- **Tracciamento end-to-end**: registrare su Blockchain gli eventi principali (creazione del pacco, richieste di ritiro, prese in carico, trasferimenti, consegne e smaltimento finale).
- **Gestione di ruoli e attori**: supportare più tipologie di attori (laboratorio, trasportatori, hub di raccolta, impianto di smaltimento) tramite dashboard dedicate e operazioni coerenti con il ruolo.
- **Integrità e auditabilità**: garantire che i dati salvati non possano essere alterati e permettere la consultazione della storia completa di un pacco tramite strumenti di ispezione.
- **Controlli automatici e prevenzione anomalie**: bloccare operazioni non valide (ad esempio trasferimenti fuori sequenza, incompatibilità di *riskCode*, superamento capacità dei trasportatori o modifiche indebite ai dati del pacco).
- **Sistema di alert**: registrare su Blockchain eventi di anomalia (`ALERT`) per rendere le violazioni tracciabili e verificabili durante controlli e audit.

3 Architettura del sistema

HazChain è organizzato secondo una architettura a più livelli, pensata per separare interfaccia utente, logica applicativa e persistenza degli eventi su Blockchain. L'obiettivo è ottenere un sistema semplice da usare, ma con una storia degli eventi immutabile e facilmente osservabile.

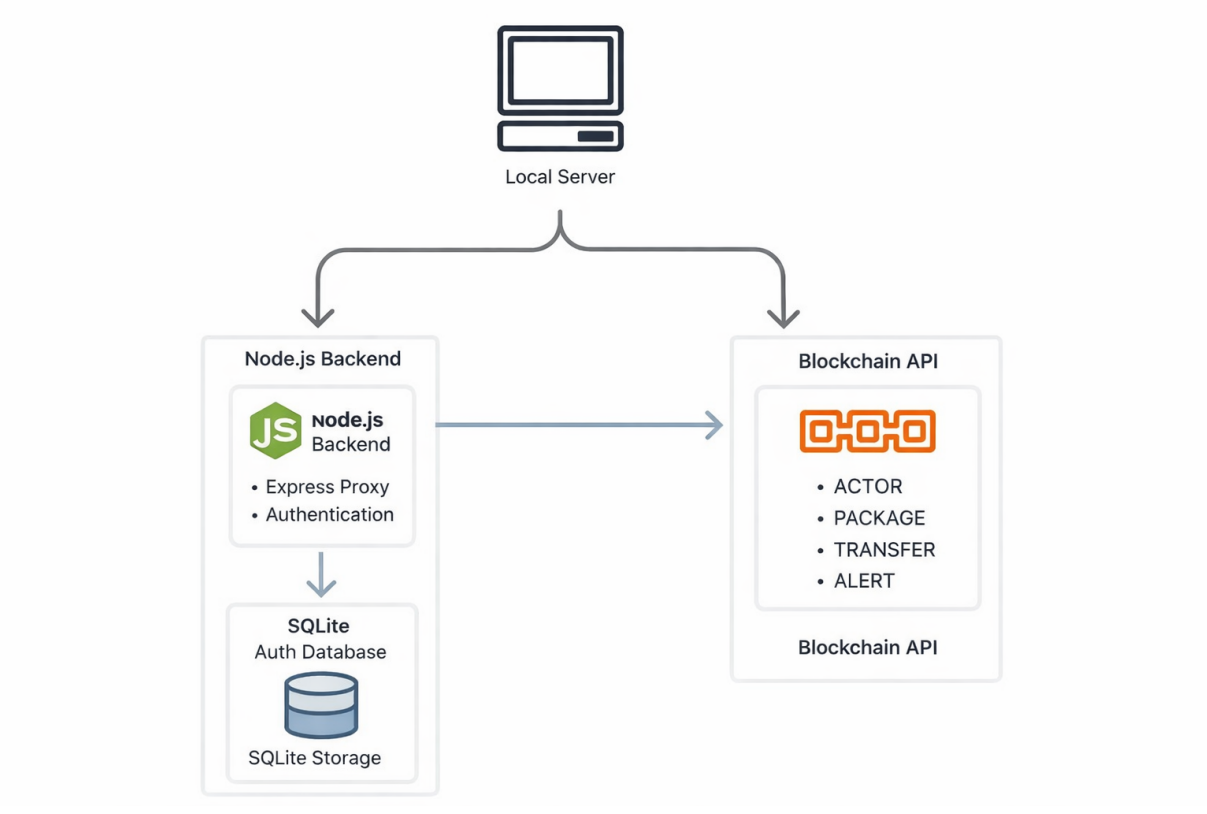

Figure 1: Schema logico dell'architettura di HazChain.

3.1 Panoramica dei componenti

Il sistema è composto dai seguenti elementi principali:

- **Frontend Web (Browser)**: insieme di pagine (dashboard) dedicate ai diversi ruoli (Lab, Van, Hub, Truck, Landfill, Alerts, strumenti di Builder/Inspector). Il frontend guida l'utente nelle operazioni, prepara le richieste e mostra lo stato dei pacchi e dei trasferimenti.
- **Backend Proxy**: espone un endpoint unico (`/api/proxy`) che riceve le richieste dal frontend e le inoltra al servizio blockchain. In alcune operazioni (in particolare sui trasferimenti) applica aggiornamenti automatici di stato del pacco.
- **Blockchain API**: servizio HTTP che offre comandi per salvare e leggere dati su Blockchain (es. inserimento e recupero chiave/valore). È il livello che garantisce persistenza immutabile e consultazione dello storico.
- **Ledger Blockchain**: registro distribuito dove vengono salvate le entità del dominio (`ACTOR`, `PACKAGE`, `TRANSFER`, `ALERT`).

3.2 Flusso delle richieste

Il flusso tipico è il seguente:

1. L'utente interagisce con una dashboard (es. crea un pacco o aggiorna un trasferimento).
2. Il frontend esegue eventuali letture preliminari (es. stato corrente del pacco) e applica controlli di validità (sequenza stati, compatibilità *riskCode*, capacità).

3. Il frontend invia una richiesta HTTP al proxy (`/api/proxy`) con un payload strutturato (comando, classe, chiavi, valore).
4. Il proxy inoltra la richiesta alla Blockchain API e restituisce al frontend l'esito.
5. Per alcune operazioni di `TRANSFER`, il proxy aggiorna anche la risorsa `PACKAGE` (es. custode corrente e stato), mantenendo coerente la chain-of-custody.

3.3 Interazione con la Blockchain API

La comunicazione con la Blockchain API avviene tramite richieste HTTP con un payload JSON. I comandi principali usati dal sistema sono:

- **`AddKV`**: inserisce/aggiorna un valore associato a una chiave.
- **`GetKV`**: recupera il valore associato a una chiave.
- **`GetKeys`**: recupera le chiavi disponibili per una classe (utile per listing e per generare nuovi id).
- **`DelKV`**: elimina dalla Blockchain una coppia chiave–valore già registrata, rimuovendo l'entità associata a quella chiave.
- **`GetKeyHistory`**: restituisce la cronologia (versioni/modifiche) associate a una chiave, utile per audit trail.
- **`GetClasses`**: restituisce l'elenco delle classi disponibili/registrate nel ledger.
- **`GetNumKeys`**: restituisce il numero di chiavi presenti per una classe (eventualmente filtrate da un prefisso).

Esempio (semplificato) di richiesta:

```
POST /api/proxy
{
  "cmd": "AddKV",
  "class": "PACKAGE",
  "key": ["PKG", "PKG-0001"],
  "value": { ... }
}
```

3.4 Modello dati su Blockchain

Le classi principali sono:

- **`ACTOR`**: rappresenta un soggetto della filiera (laboratorio, trasportatore, hub, discarica) con attributi come ruolo, capacità e *riskCodes* gestibili.
- **`PACKAGE`**: rappresenta il pacco di rifiuto (tipo, peso, *riskCode*), con stato e custode corrente.
- **`TRANSFER`**: rappresenta un evento di passaggio di custodia, con azione e stato (`WAITING`, `SHIPPING`, `COMPLETED`).
- **`ALERT`**: registra anomalie o tentativi di operazioni non valide, rendendole auditabili.

3.5 Workflow operativo

Il processo logico è fatto così:
`LAB` → `VAN` → `HUB` → `TRUCK` → `LANDFILL` → `DISPOSED`
Ogni transizione viene registrata tramite `TRANSFER` e porta all'aggiornamento dello stato del `PACKAGE` (custode e posizione logica), consentendo di ricostruire in modo affidabile la storia completa.

4 Struttura del codice

Il progetto **HazChain** è organizzato in due macro-componenti: **frontend** (interfaccia web e logica lato browser) e **backend** (proxy Node/Express che espone il sito e inoltra le chiamate alla Blockchain API).

4.1 Frontend

Il frontend è composto da pagine HTML (una per ruolo) che condividono:

- **Accesso** (`login.html`): la registrazione e il login sono gestiti nella pagina `login.html`. Le dashboard operative effettuano un controllo di sessione lato client (se non autenticato, redirect a `login.html`).
- una sezione **Builder** (form dinamica) per eseguire azioni e scrivere eventi su Blockchain;
- una sezione **Inspector** per cercare un `PACKAGE` e visualizzarne le informazioni;
- un **System Log** che mostra esito delle operazioni e dettagli delle risposte API.

Pagine principali

- `login.html`: interfaccia con form di Login e Registrazione e logica di redirect in base al ruolo.
- `lab.html`, `van.html`, `hub.html`, `truck.html`, `landfill.html`: dashboard dedicate che espongono solo i pulsanti/azioni coerenti con il ruolo.
- `alerts.html`: pagina di monitoraggio che mostra gli `ALERT` presenti sul ledger.

`style.css` Contiene tutto lo stile dell'interfaccia (layout, card, form, log console, badge di stato, ecc.) e gli elementi grafici del "monitoring wall".

`ui.js` (UI e rendering) Questo file contiene la logica di interfaccia UI:

- **Logger**: funzione `log(...)` per scrivere eventi nel pannello "System Log".
- **Builder dinamico**: costruzione delle righe attributo (input, select, checkbox-group), gestione campi *read-only* per attributi di sistema e merge tra attributi visibili e "hidden".
- **Template attori**: pulsanti che precompilano la form per creare `ACTOR` (LAB, VAN/F, HUB, TRUCK/C, LANDFILL) fissando `role` come attributo nascosto.
- **Inspector rendering**: funzioni per visualizzare in modo leggibile lo stato di un package (card + badge per `state`).
- **Package ID lookup**: mini-form usata quando l'utente avvia un'azione senza avere un package già selezionato nel builder.

`logic.js` (business logic e flussi) Questo file contiene la logica applicativa e i flussi di processo:

- **API wrapper**: `callBlockchain(payload, actionName)` invia richieste HTTP al proxy (`/api/proxy`) e gestisce le risposte.
- **Fetch helpers**: `fetchPackageState`, `fetchTransferState`, `fetchActorState` per leggere lo stato dal ledger.
- **Generazione ID**: `generateNextId(...)` usa `GetKeys` e incrementa il contatore per creare ID del tipo: `PKG-2026-0001`, `TR-F-0001`, `TR-C-0001`, `AL-0001`.
- **Validazioni**: controlli prima di scrivere un `TRANSFER`:
  - integrità dei campi critici (peso/tipologia/riskCode coerenti col `PACKAGE`);
  - compatibilità `riskCode` con `riskCodesHandled` dell'attore destinatario;
  - validazione della **sequenza** degli stati;
  - vincoli di **capacità** del mezzo (kg trasportabili).

  In caso di violazione l'azione viene bloccata e viene registrato un `ALERT`.
- **Binding eventi**: associa i pulsanti delle dashboard alle azioni dei flussi (es. Create Package, Request Pickup, Pickup/Arrive, Dispose).

4.2 Backend

Il backend è un servizio **Node.js/Express** con tre responsabilità principali:

- **Hosting statico**: serve i file in `public/` e rende disponibile l'app su `http://localhost:3000` (entrypoint: `login.html`).
- **Proxy verso Blockchain API**: endpoint `POST /api/proxy` che inoltra il payload a `http://localhost:9999/api` per leggere/scrivere le entità su Blockchain (`ACTOR`, `PACKAGE`, `TRANSFER`, `ALERT`).
- **Autenticazione**: endpoint `POST /api/register` (creazione utente) e `POST /api/login` (verifica credenziali e rilascio token) basati su database locale.

`index.js` (proxy con auto-sync e auth) Oltre a inoltrare le richieste verso la Blockchain API, il server:

- espone gli endpoint di autenticazione `/api/register` e `/api/login`;
- intercetta le operazioni `AddKV` su `TRANSFER` e esegue un **aggiornamento automatico** del relativo `PACKAGE`:
  - aggiorna `currentCustodian` in base a `from/to` e `status`;
  - aggiorna `state` in modo coerente (es. `F_TRANSPORT`, `IN_HUB`, `C_TRANSPORT`, `IN_LANDFILL`);
  - imposta `lastTransferId` e `lastUpdateTs`.

`db.js` Gestisce l'inizializzazione del database SQLite e la creazione della tabella `users` (utilizzata per persistere `username`, `password_hash` e `role`).

`package.json` Definisce le dipendenze del backend e gli script di avvio in sviluppo (es. `nodemon`). Oltre ai moduli di base (Express, Axios, CORS, Morgan), include anche le librerie per l'autenticazione: `sqlite3` (persistenza utenti), `bcrypt` (hash/verifica password) e `jsonwebtoken` (generazione token).

5 Interfacce utente

L'obiettivo è ridurre gli errori operativi: le dashboard sono separate per ruolo, ma l'enforcement è applicato sulle operazioni tramite controlli lato client (azioni non autorizzate vengono bloccate e registrate come `ALERT`).
Le **dashboard operative** (`lab.html`, `van.html`, `hub.html`, `truck.html`, `landfill.html`) condividono tre elementi principali:

- **Area operativa (azioni guidate)**: pulsanti e form per eseguire le operazioni consentite dal flusso.
- **Inspector**: ricerca di un `PACKAGE` e visualizzazione di stato corrente, custode e riferimenti all'ultimo trasferimento.
- **System Log**: diagnostica delle operazioni e delle risposte della Blockchain API.

La pagina `alerts.html` è invece dedicata al **monitoraggio** degli `ALERT` e non include Inspector/Builder.

5.1 Dashboard disponibili e responsabilità

Di seguito una sintesi delle dashboard e delle operazioni principali supportate:

- **Lab** (`lab.html`): creazione di nuovi `PACKAGE` e apertura di un `TRANSFER` in stato `WAITING` per richiedere il ritiro al Van.
- **Van** (`van.html`): gestione del trasporto leggero. Consente pickup (passaggio a `SHIPPING`) e consegna a Hub (passaggio a `COMPLETED`).
- **Hub (`hub.html`)**: punto di stoccaggio intermedio. Permette l'ispezione dei pacchi ricevuti e la richiesta di ritiro verso Truck (creazione `TRANSFER` in stato `WAITING`).
- **Truck** (`truck.html`): trasporto di rifiuti pericolosi. Consente pickup da Hub e consegna a Landfill, applicando i controlli di compatibilità e capacità.
- **Landfill** (`landfill.html`): chiusura del processo con ricezione e marcatura del pacco come `DISPOSED`.
- **Storico** (`history.html`): pagina che mostra lo storico di un Package indicato.
- **Alerts** (`alerts.html`): pagina di monitoraggio che mostra gli `ALERT` registrati sul ledger, utile per audit e diagnosi di anomalie.

5.2 Pagina di autenticazione (Login/Registrazione)

L'accesso avviene tramite la pagina `login.html`, che presenta due form: **Login** e **Registrazione**. In caso di login riuscito, l'utente viene reindirizzato automaticamente alla dashboard coerente con il proprio ruolo. La sessione applicativa viene mantenuta tramite salvataggio locale delle informazioni di accesso (*token* e dati utente); se l'utente risulta già autenticato, la pagina effettua direttamente il redirect alla dashboard corretta.

5.3 Sequenze principali da documentare

Per descrivere in modo chiaro il comportamento del sistema, sono stati individuati sette flussi chiave, rappresentati tramite sequence diagram.

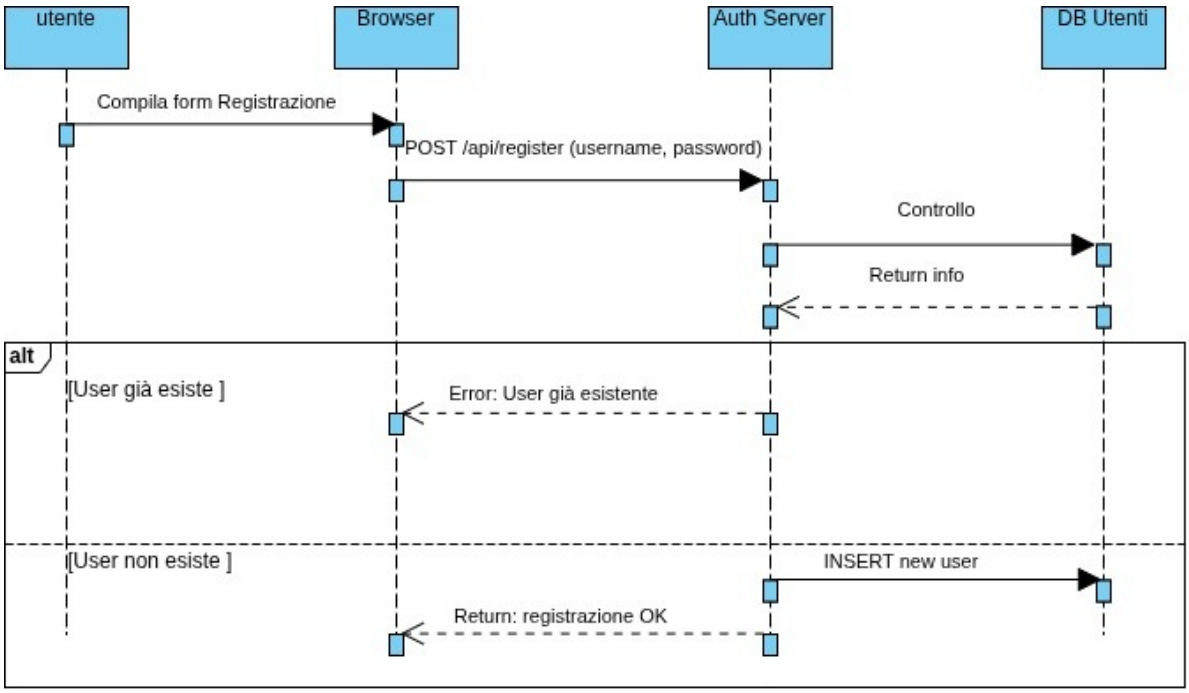

Figure 2: Registrazione.

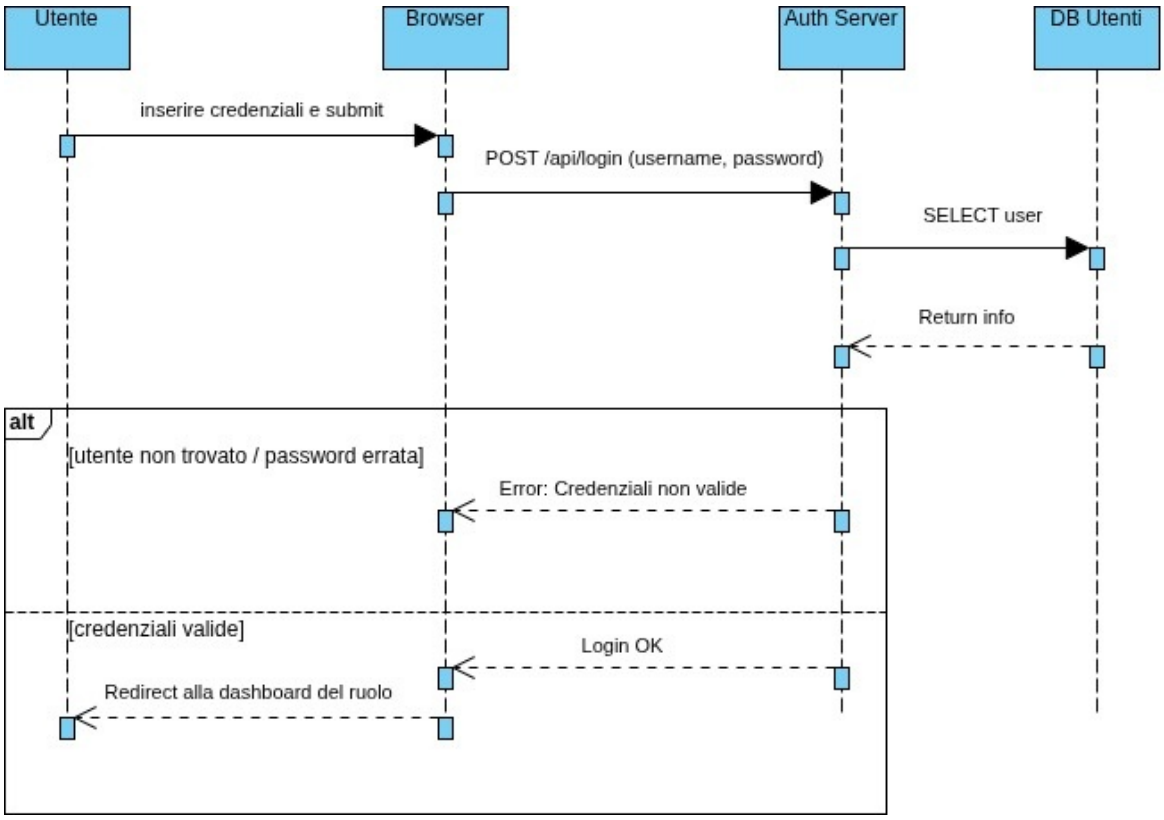

Figure 3: Log-in.

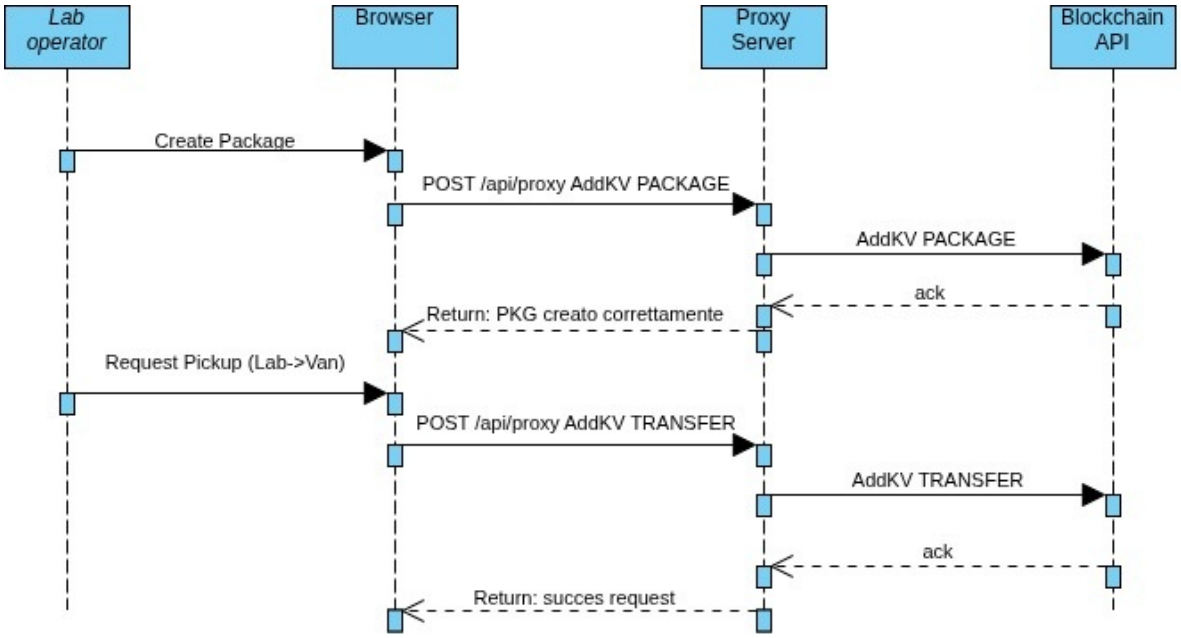

Figure 4: Create PKG, Request Pickup.

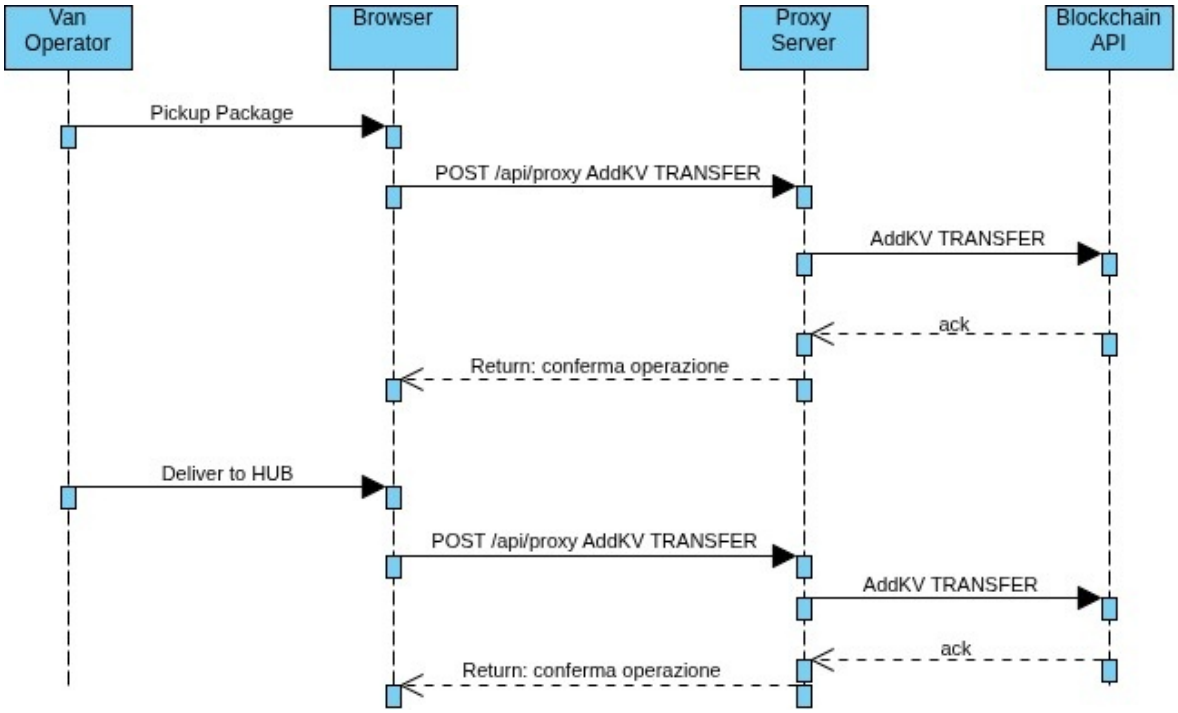

Figure 5: Van to Hub.

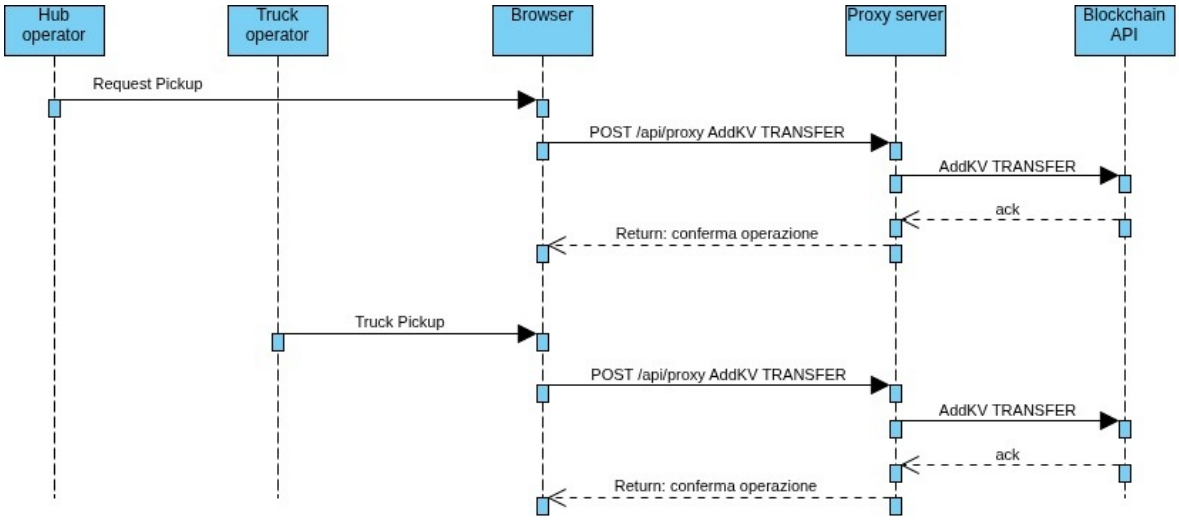

Figure 6: Hub to Landfill.

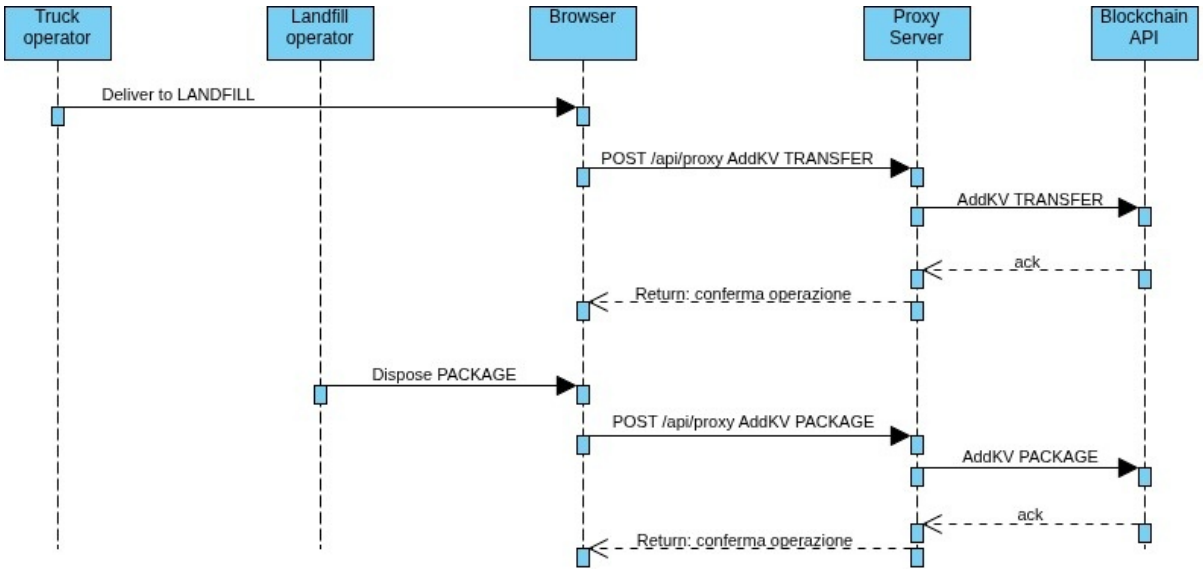

Figure 7: Truck to Dispose.

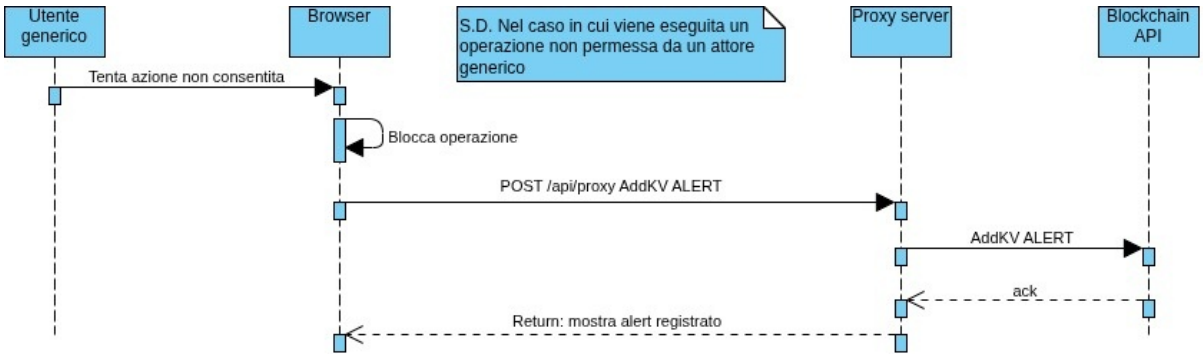

Figure 8: Alert.

6 Funzionalità del sistema HazChain

Questa sezione descrive le principali funzionalità del sistema, seguendo il flusso operativo della *chain-of-custody*. Ogni operazione genera eventi persistiti su Blockchain sotto forma di entità `ACTOR`, `PACKAGE`, `TRANSFER` e `ALERT`.

6.1 Autenticazione utente (Registrazione e Login)

Il sito introduce un meccanismo di autenticazione basato su credenziali `username/password`. La registrazione crea un nuovo utente nel database locale, mentre il login restituisce un token che abilita l'accesso alle dashboard operative. Una volta autenticato, l'utente viene reindirizzato alla pagina coerente con il proprio ruolo.

- **Registrazione** Durante la registrazione l'utente invia `username`, `password` e `role`. Il server verifica che lo username non sia già presente e, in caso positivo, salva il nuovo utente nel database.
- **Login** Durante il login l'utente invia `username` e `password`. Il server verifica le credenziali e, se valide, restituisce un token e le informazioni del ruolo. Il browser salva i dati e reindirizza l'utente alla dashboard del ruolo.

Nota sulla registrazione nel prototipo (modalità test) La funzionalità di **registrazione** presente nella versione attuale dell'applicazione è pensata principalmente per **test e dimostrazione** del sistema. In particolare, la possibilità di selezionare liberamente il `role` in fase di registrazione (incluso `SUPERUSER`) **non** è una scelta ammissibile in un contesto reale, poiché consentirebbe a chiunque di ottenere privilegi elevati e di aggirare i controlli applicativi. Nella versione finale, la creazione degli account e l'assegnazione dei ruoli verranno gestite in modo **controllato** impedendo la registrazione autonoma con privilegi arbitrari.

Nota su ruoli e appartenenza Nel prototipo attuale l'account utente è associato ad un **ruolo** (LAB, VAN, HUB, TRUCK, LANDFILL) ma **non** è vincolato ad una specifica istanza fisica (es. un laboratorio particolare o uno specifico mezzo). In altre parole, il login identifica il tipo di operatore e abilita le azioni coerenti con quel ruolo, senza introdurre un legame univoco con un singolo `ACTOR` registrato sul ledger. Per esempio una persona autenticata come LAB Operator potrà creare un package per qualsiasi laboratorio esistente nella classe `ACTOR`.

6.1.1 Funzionamento algoritmico

Dal punto di vista algoritmico, l'autenticazione è gestita in due fasi.

Registrazione: il client invia `{username, password, role}` a `POST /api/register`; il server valida che i campi siano presenti e interroga SQLite per verificare che lo `username` non esista già; se lo username è libero, il server calcola l'hash della password con `bcrypt` (utilizzando dei salt rounds) e salva nel DB **solo** `password_hash` (mai la password in chiaro) insieme a `username` e `role`; restituisce esito di creazione utente.

Login: il client invia `{username, password}` a `POST /api/login`; il server recupera l'utente dal DB tramite `username` e confronta la password inserita con l'hash salvato usando `bcrypt.compare`; se il confronto fallisce ritorna errore, altrimenti genera un token con payload `{id, username, role}` e scadenza `24h`; il browser salva `token` e `user` in `localStorage` e reindirizza alla dashboard associata al ruolo (es. LAB→`lab.html`, HUB→`hub.html`, ecc.). Le pagine operative, all'avvio, verificano la presenza di `token` e `user` in `localStorage` e in caso contrario reindirizzano automaticamente a `login.html`; il logout elimina tali valori.

6.2 Creazione attori (`ACTOR`)

Il sistema consente di registrare gli attori della filiera (Lab, Van, Hub, Truck, Landfill) specificando ruolo e attributi operativi (es. capacità, *riskCodesHandled*). La creazione produce un record `ACTOR` sul ledger e abilita l'attore a partecipare ai trasferimenti.

6.3 Creazione pacco (`PACKAGE`)

Il laboratorio crea un nuovo pacco di rifiuto inserendo i dati principali (tipo rifiuto, peso, *riskCode*). Il sistema genera un identificativo univoco e registra un `PACKAGE` con stato iniziale (es. `IN_LAB`) e custode corrente impostato al laboratorio.

6.4 Richiesta ritiro Lab → Van (`TRANSFER: WAITING`)

Il laboratorio apre una richiesta di ritiro creando un `TRANSFER` in stato `WAITING`, che indica che il pacco è pronto per essere preso in carico dal Van. La richiesta lega pacco, mittente e destinatario.

6.5 Pickup e trasporto Van → Hub (`TRANSFER: SHIPPING/COMPLETED`)

Il Van prende in carico il pacco aggiornando il trasferimento a `SHIPPING`. Alla consegna presso l'Hub, il trasferimento viene marcato `COMPLETED` e il `PACKAGE` viene aggiornato (nuovo custode e stato).

6.6 Richiesta ritiro Hub → Truck (`TRANSFER: WAITING`)

L'Hub crea un `TRANSFER` in stato `WAITING` verso un Truck, indicando che il pacco è pronto per il trasporto di rifiuti pericolosi verso la destinazione finale.

6.7 Pickup e trasporto Truck → Landfill (`TRANSFER: SHIPPING/COMPLETED`)

Il Truck effettua il pickup (stato `SHIPPING`) e completa la consegna alla Landfill (stato `COMPLETED`). Anche in questo caso il `PACKAGE` viene aggiornato per riflettere il custode corrente e lo stato.

6.8 Smaltimento finale (`PACKAGE: DISPOSED`)

La Landfill esegue l'operazione di smaltimento che marca il `PACKAGE` come `DISPOSED`, chiudendo formalmente il ciclo di vita del pacco e rendendo non più ammessi ulteriori trasferimenti operativi.

6.9 Inspector

L'Inspector permette di cercare un pacco e ricostruire la sua storia completa: stato corrente, custode attuale, ultimo trasferimento. Questa funzione supporta verifiche e audit in modo rapido e ripetibile.

6.10 Storico del Package

Oltre alla visualizzazione dello stato corrente, c'è una funzionalità dedicata alla consultazione dello **storico completo** di un `PACKAGE`. L'utente inserisce il `packageId` e il frontend invoca il comando `GetKeyHistory` sulla chiave del pacco, ottenendo la sequenza delle versioni/modifiche registrate nel tempo.

6.11 `ALERT` e regole di blocco

Prima di registrare eventi critici, il sistema applica regole di validazione (sequenza degli stati, integrità dei dati, compatibilità del *riskCode*, rispetto della capacità). In caso di violazione l'operazione viene bloccata e viene registrato un `ALERT` su Blockchain, rendendo l'anomalia tracciabile e verificabile.

Tipologie di `ALERT` implementate Nell'applicazione sono presenti le seguenti tipologie di alert:

- `SEQUENCE_VIOLATION`: viene generato quando un'azione viene tentata con il `PACKAGE` in uno stato non coerente con il flusso previsto (es. consegna senza pickup).
  Campi principali: `packageId`, `currentState`, `requiredStates`, `attemptedAction`.
- `INTEGRITY_MISMATCH`: viene generato quando un `TRANSFER` tenta di portare valori non coerenti con quelli del `PACKAGE` (es. mismatch su campi critici come peso/tipologia/riskCode).
  Campi principali: `packageId`, `transferId`, `mismatches`.
- `RISK_CODE_INCOMPATIBLE`: viene generato quando il destinatario del trasferimento non è abilitato a gestire i *riskCode* del pacco (assenza di sovrapposizione tra codici del pacco e `riskCodesHandled` dell'attore).
  Campi principali: `packageId`, `transferId`, `actorId`, `packageRiskCodes`, `actorRiskCodesHandled`.
- `CAPACITY_EXCEEDED`: viene generato quando un trasportatore supererebbe la propria capacità massima caricando il pacco (vincolo su `capacityKg`).
  Campi principali: `vehicleId`, `packageId`, `capacityKg`, `currentLoadKg`, `packageWeightKg`, `attemptedLoadKg`.
- `UNAUTHORIZED_ACTION`: viene generato quando un utente autenticato tenta un'operazione non consentita dal proprio ruolo (controllo accessi logico lato applicazione).
  Campi principali: `userId`, `userRole`, `requiredRole`, `attemptedAction`.
- `DUPLICATE_ENTITY`: viene generato quando si tenta di creare un'entità con un ID già esistente (controllo su `PACKAGE` in creazione e su `ACTOR`).
  Campi principali: `entityClass`, `entityId` e, a seconda del caso, `existingState` (per `PACKAGE`) oppure `existingRole` (per `ACTOR`).

7 Requisiti

7.1 Requisiti funzionali

- **RF1 — Gestione attori**: il sistema deve permettere la creazione e la consultazione di `ACTOR` (Lab, Van, Hub, Truck, Landfill) con attributi come ruolo, capacità e *riskCodesHandled*.
- **RF2 — Creazione pacco**: il laboratorio deve poter creare un nuovo `PACKAGE` con ID univoco e dati (wasteType, weightKg, riskCode), impostando stato iniziale e custode. **Non** è possibile in alcun modo modificare il Package una volta creato (come separazione Package in due Package diversi, o diminuzione del peso).
- **RF3 — Richiesta ritiro**: il laboratorio deve poter creare un `TRANSFER` a `WAITING` verso un Van.
- **RF4 — Pickup/Consegna Van**: il Van deve poter aggiornare il `TRANSFER` a `SHIPPING` (pickup) e a `COMPLETED` (consegna a Hub), con conseguente aggiornamento del `PACKAGE`.

- **RF5 — Richiesta ritiro**: l'Hub deve poter creare un `TRANSFER` a `WAITING` verso un Truck.
- **RF6 — Pickup/Consegna Truck**: il Truck deve poter eseguire pickup e consegna verso Landfill (stati `SHIPPING`/`COMPLETED`), aggiornando il `PACKAGE`.
- **RF7 — Smaltimento**: la Landfill deve poter marcare il `PACKAGE` come `DISPOSED` chiudendo il ciclo.
- **RF8 — Inspector**: il sistema deve permettere la ricerca di un pacco e la visualizzazione delle informazioni riguardante esso.
- **RF9 — Alert**: il sistema deve registrare `ALERT` su Blockchain in caso di anomalie o operazioni non valide.

7.2 Requisiti non funzionali e sicurezza

- **RNF1 — Usabilità**: l'interfaccia deve essere semplice e coerente tra dashboard, riducendo errori operativi (azioni guidate per ruolo).
- **RNF2 — Affidabilità**: il sistema deve gestire errori di rete o API (timeout, risposte non valide) mostrando messaggi chiari nel System Log.
- **RNF3 — Tracciabilità**: tutte le operazioni critiche devono essere persistite su Blockchain per garantire auditabilità e ricostruzione della chain-of-custody.
- **RNF4 — Integrità dei dati**: prima di scrivere eventi devono essere applicati controlli, in caso di violazione l'operazione viene bloccata. (es. modifiche a campi critici del `PACKAGE` o transizioni di stato fuori sequenza).
- **RNF5 — Controllo accessi (logico)**: le dashboard devono limitare le azioni disponibili in base al ruolo (Lab/Van/Hub/Truck/Landfill).
- **RNF6 — Non ripudio**: gli eventi registrati sul ledger devono risultare immutabili e verificabili, rendendo evidenti eventuali anomalie tramite `ALERT`.
- **RNF7 — Privacy minimale**: la piattaforma deve evitare dati personali non necessari, limitandosi a informazioni operative (ID, ruolo, parametri tecnici del pacco).

Controlli di sicurezza lato server Per aumentare la sicurezza del sistema, l'accesso a `/api/proxy` è protetto tramite token JWT e i controlli principali sono stati spostati dal client al server. Questo approccio è più sicuro perché impedisce di fidarsi del browser. Anche se un utente modifica il codice JavaScript o il payload delle richieste, il backend continua a verificare ruolo, permessi e validità dell'operazione prima di inoltrarla alla Blockchain API.

8 Stakeholder

Gli stakeholder di **HazChain** sono i soggetti coinvolti, a vario titolo, nel processo di gestione e tracciamento dei rifiuti tossici e nelle attività di controllo.

- **Laboratorio (LAB)**: produce i rifiuti, crea i `PACKAGE` e avvia le richieste di ritiro.
- **Trasportatore leggero (VAN / F-Transport)**: effettua il ritiro dal laboratorio e la consegna presso l'Hub.

- **Centro di raccolta / Hub (HUB)**: riceve e stocca temporaneamente i pacchi, verifica lo stato e avvia le richieste di ritiro verso il Truck.
- **Trasportatore rifiuti pericolosi (TRUCK / C-Transport)**: prende in carico i pacchi dall'Hub e li consegna all'impianto finale di smaltimento.
- **Impianto di smaltimento / Discarica (LANDFILL)**: riceve i pacchi e chiude il ciclo di vita marcandoli come `DISPOSED`.
- **Ispettore / Autorità di controllo**: consulta l'audit trail, verifica la coerenza della chain-of-custody e analizza eventuali `ALERT` registrati sul ledger.

9 Considerazioni finali e sviluppi futuri

Il progetto dimostra come una applicazione web supportata da un registro immutabile possa migliorare il tracciamento della *chain-of-custody* dei rifiuti tossici. La registrazione degli eventi su Blockchain rende più semplice ricostruire lo storico dei pacchi, riduce il rischio di manomissioni e facilita le attività di audit. Inoltre, l'introduzione di validazioni e `ALERT` rende più evidente e verificabile la presenza di operazioni incoerenti o potenzialmente fraudolente.
Come sviluppi futuri, **HazChain** potrebbe essere esteso in diverse direzioni:

- **Identità e firme digitali**: firma dele operazioni (attore che scrive l'evento) per rafforzare non ripudio e attribuzione, anche lato blockchain.
- **Integrazione con sensori/IoT**: QR code o RFID sui pacchi e acquisizione automatica di eventi (timestamp, geolocalizzazione, temperatura, integrità del contenitore).
- **Dashboard avanzate**: statistiche, reportistica e monitoraggio in tempo reale (volumi, tempi medi, alert per attore/area).